Strategy

Strategy · Swap cart pricing (percent vs flat vs half) · Build with Waqar

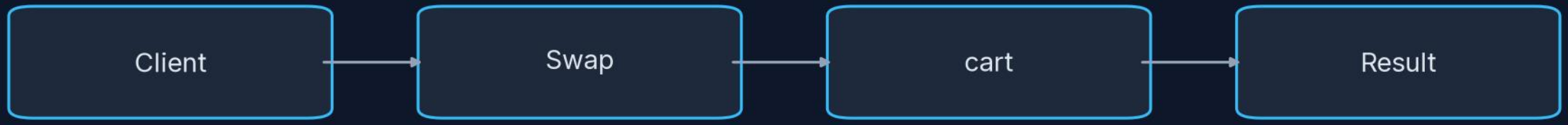
Today's Hook

- Problem: Swap cart pricing (percent vs flat vs half)
- Pattern: Strategy — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Swap cart pricing (percent vs flat vs half)
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Strategy: Swap cart pricing (percent vs flat vs half)

Real-World Scenario

- Running example: Swap cart pricing (percent vs flat vs half)
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.strategy.problem;

import java.util.*;

public final class NaiveCart {
  public static int total(int cents, String mode){
    if("pct".equals(mode)) return cents*90/100;
    if("flat".equals(mode)) return Math.max(0,cents-200);
    return cents/2;
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.gof.behavioral.strategy.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.function.*;

interface Pricing { int apply(int cents); }
final class PercentOff implements Pricing { private final int p; PercentOff(int p){this.p=p;} public int apply(int c){ return c*(100-p)/100; } }
final class FlatOff implements Pricing { private final int f; FlatOff(int f){this.f=f;} public int apply(int c){ return Math.max(0,c-f); } }
final class HalfOff implements Pricing { public int apply(int c){ return c/2; } }
final class Cart {
  private final Pricing pricing; Cart(Pricing p){ pricing=p; }
  int total(int cents){ return pricing.apply(cents); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.behavioral.strategy.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Strategy: cart pricing ===");
    System.out.println("--- naive ---"); System.out.println(NaiveCart.total(1000,"pct"));
System.out.println("--- strategy ---");
System.out.println(new Cart(new PercentOff(10)).total(1000));
System.out.println(new Cart(new FlatOff(200)).total(1000));
System.out.println(new Cart(new HalfOff()).total(1000));
  }
}
```

Walkthrough Output

```
=== Strategy: cart pricing ===
--- naive ---
900
--- strategy ---
900
800
500
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Strategy.

Related Patterns

- Contrast Strategy with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Strategy → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.